Step 4 — Stream the Reply Live and Build One Structured Record per Ticket — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: one reply is actually needed in two different shapes at once. A support agent watching the screen wants to see it **as it's being written**, via `.stream()`. The ticketing system wants a **checked record** — category, reply, whether the fallback was used, whether it needs a human — to store and route on. Both must be built from the exact same reply, not two separate model calls that could disagree, and the unclear ticket from Step 1 plus every fallback from Step 3 must now be flagged for a person to actually look at, not quietly auto-decided. **If not:** the agent stares at a blank screen while waiting, and the ambiguity Step 1 revealed disappears without a trace.

After this step the project is complete: every reply streams live, and every ticket ends with a record that says whether a human should look at it.

Solution — a `TicketRecord`, a testable review rule, and a streaming helper

Changed from Step 3: `TicketRecord` adds two fields, both with defaults.

Story — `schemas.py`: `needs_human_review` is the boolean flag a real ticketing system would route on; `review_reasons` is a list that names **which specific rule** fired, not just that something did. **If not:** a person would see a bare `True` on a flagged ticket and have to guess why it was flagged.

```
# src/schemas.py
# The pipeline's checked shapes. TicketRecord now also carries the human
# review flag and the reasons; the defaults keep earlier code working.
from typing import Literal

from pydantic import BaseModel

class TicketCategory(BaseModel):
    category: Literal["billing", "technical", "general"]
    reason: str
    sure: bool

class TicketRecord(BaseModel):
```

```
    customer_id: str
    ticket: str
    category: str
    reply: str
    used_fallback: bool = False
    needs_human_review: bool = False        # new in Step 4
    review_reasons: list[str] = []          # new in Step 4
```

Changed from Step 3: `handle_ticket()` now streams the reply and fills the two new fields. Its name, inputs and return type stay the same.

Story — `services/pipeline.py`: `stream_reply()` prints each chunk as it arrives via `.stream()` and also collects and returns the full text once done; `review_reasons()` holds the flagging rules as plain, testable Python (fallback was used, or the classifier wasn't sure); `handle_ticket()` builds the final record from the **exact same** streamed reply the customer already saw, never a second call. **If not:** a second model call made just to build the record could quietly say something different from the reply the customer actually read.

```
# src/services/pipeline.py
# handle_ticket(): classify, reply (streamed live), build the record. New
# here: stream_reply() prints the reply as it arrives, and review_reasons()
# decides when a human should look at the ticket.
from langchain_core.runnables import Runnable     # new in Step 4

from src.chains.classifier import classify_ticket
from src.chains.fallback import FALLBACK_REPLY
from src.chains.replies import build_reply_chain
from src.schemas import TicketRecord

# new in Step 4
def stream_reply(chain, inputs):
    print("Reply: ", end="", flush=True)
    reply = ""
    for chunk in chain.stream(inputs):
        print(chunk, end="", flush=True)
        reply = reply + chunk
    print()
    return reply

# new in Step 4
def review_reasons(category, used_fallback):
    reasons = []
    if used_fallback:
        reasons.append("the fallback reply was sent")
    if not category.sure:
        reasons.append("the ticket could fit more than one category")
```

```
    return reasons

def handle_ticket(ticket, customer_id):
    category = classify_ticket(ticket)
    chain = build_reply_chain(category.category)
    inputs = {"ticket": ticket, "customer_id": customer_id}
    reply = stream_reply(chain, inputs)     # changed in Step 4: .stream()

    used_fallback = reply == FALLBACK_REPLY
    reasons = review_reasons(category, used_fallback)     # new in Step 4

    return TicketRecord(
        customer_id=customer_id,
        ticket=ticket,
        category=category.category,
        reply=reply,
        used_fallback=used_fallback,
        needs_human_review=len(reasons) > 0,     # new in Step 4
        review_reasons=reasons,                  # new in Step 4
    )
```

Changed from Step 3: `print_record()` adds the review flag and its reasons; the reply itself was already streamed.

Story — `main.py`: the loop simply displays results now; any review reasons for a ticket are listed right under it, visibly. **If not:** a flagged ticket would print identically to a clean one, defeating the whole point of flagging it.

```
# main.py
# The file you run: python main.py
# Loads the config, reads the sample tickets, runs handle_ticket() on each,
# and prints each result with print_record().
import json
import sys

import openai

from src.config import load_config
from src.exceptions import MissingConfigError
from src.services.pipeline import handle_ticket
from src.utils.logger import get_logger

def load_samples():
    with open("data/sample/tickets.json") as samples_file:
        return json.loads(samples_file.read())
```

```
def print_record(record):
    print(f"  category:      {record.category}")
    print(f"  used fallback: {record.used_fallback}")
    print(f"  human review:  {record.needs_human_review}")     # new in Step 4
    for reason in record.review_reasons:                       # new in Step 4
        print(f"    - {reason}")

def main():
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    samples = load_samples()
    print(f"LangChainPro — {len(samples)} tickets in the queue.")

    for sample in samples:
        print()
        print(f"Ticket from {sample['customer_id']}: {sample['ticket']}")
        try:
            record = handle_ticket(sample["ticket"], sample["customer_id"])
        except openai.AuthenticationError:
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        print_record(record)

if __name__ == "__main__":
    main()
```

Story — the tests: `test_review.py` checks each flagging rule on its own with hand-made inputs, no API key needed, since the rules are plain Python. `test_schemas.py` is unchanged from Step 1: its old record still builds fine, because the new fields all have defaults. The integration test now adds a real check that the deliberately unclear ticket genuinely gets flagged by the live pipeline. **If not:** a flagging rule could silently stop firing and nobody would notice until a ticket slipped through unreviewed.

```
# tests/unit/test_review.py
# Checks the "needs human review" rules with hand-made inputs. Each case
# prints OK or WRONG. No API key needed.
# Run from the repo root: python -m tests.unit.test_review
from src.schemas import TicketCategory
from src.services.pipeline import review_reasons
```

```
def make_category(sure):
    return TicketCategory(category="billing", reason="test", sure=sure)

def test_clear_ticket():
    reasons = review_reasons(make_category(True), False)
    ok = "OK"
    if reasons != []:
        ok = "WRONG"
    print(f"case 1 (sure, no fallback) -> {ok}")

def test_unsure_ticket():
    reasons = review_reasons(make_category(False), False)
    ok = "OK"
    if len(reasons) != 1:
        ok = "WRONG"
    print(f"case 2 (unsure flagged)   -> {ok}")

def test_fallback_used():
    reasons = review_reasons(make_category(True), True)
    ok = "OK"
    if len(reasons) != 1:
        ok = "WRONG"
    print(f"case 3 (fallback flagged) -> {ok}")

if __name__ == "__main__":
    test_clear_ticket()
    test_unsure_ticket()
    test_fallback_used()
```

Changed from Step 1: the integration test adds case 5 — the unclear ticket must be flagged for a human.

```
# tests/integration/test_live_pipeline.py
# Runs the classifier and the whole pipeline against the real API — each
# step adds cases. Each case prints OK or WRONG. Needs OPENAI_API_KEY.
# Run from the repo root: python -m tests.integration.test_live_pipeline
import json

from src.chains.classifier import classify_ticket
from src.config import load_config
from src.services.pipeline import handle_ticket

def load_samples():
    with open("data/sample/tickets.json") as samples_file:
        return json.loads(samples_file.read())
```

```
def test_billing_classified(samples):
    result = classify_ticket(samples[0]["ticket"])
    ok = "OK"
    if result.category != "billing":
        ok = "WRONG"
    print(f"case 1 (billing)   -> {ok} ({result.category})")

def test_technical_classified(samples):
    result = classify_ticket(samples[1]["ticket"])
    ok = "OK"
    if result.category != "technical":
        ok = "WRONG"
    print(f"case 2 (technical) -> {ok} ({result.category})")

def test_general_classified(samples):
    result = classify_ticket(samples[2]["ticket"])
    ok = "OK"
    if result.category != "general":
        ok = "WRONG"
    print(f"case 3 (general)   -> {ok} ({result.category})")

def test_full_pipeline(samples):
    record = handle_ticket(samples[0]["ticket"], samples[0]["customer_id"])
    ok = "OK"
    if record.category != "billing" or record.reply == "":
        ok = "WRONG"
    print(f"case 4 (full pipeline) -> {ok}")

# new in Step 4
def test_unclear_flagged(samples):
    record = handle_ticket(samples[3]["ticket"], samples[3]["customer_id"])
    ok = "OK"
    if not record.needs_human_review:
        ok = "WRONG"
    print(f"case 5 (unclear flagged) -> {ok}")

if __name__ == "__main__":
    load_config()
    samples = load_samples()
    test_billing_classified(samples)
    test_technical_classified(samples)
    test_general_classified(samples)
    test_full_pipeline(samples)
    test_unclear_flagged(samples)
```

Run it (from the repo root):

```
python -m tests.unit.test_review
python -m tests.unit.test_schemas
python main.py
python -m tests.integration.test_live_pipeline
```

`test_review` prints:

```
case 1 (sure, no fallback) -> OK
case 2 (unsure flagged)   -> OK
case 3 (fallback flagged) -> OK
```

`python main.py` ends the unclear ticket (`C-1004`) like this (log lines left out; model replies vary):

```
Ticket from C-1004: My invoice and the dashboard numbers don't match. Bug?
Reply: Sorry about the mismatch. Our billing team will check your
invoice within 2 business days.
  category:      billing
  used fallback: False
  human review:  True
    - the ticket could fit more than one category
```